

TP noté n°1 2024	: Système d'exploitation
Durée	: 1h45
Documents autorisés	: Le poly système uniquement
Enseignante	: Fetia Bannour

Exercice 1

1. Indiquez ce que fait la commande suivante :

```
bash> find /var/log -type f -name '*.log' -exec stat -c '%s %n' {} \; |
sort -n | tail -n 3
```

2. Indiquez ce que fait la commande suivante :

```
bash> ps aux | grep '[h]ttpd' | { sum=0; while read -r line; do
    mem=$(echo "$line" | sed 's/ */ /g' | cut -d ' ' -f4)
    sum=$((sum + ${mem%.*})); done
    echo "$sum%"
}
```

3. Dans une arborescence de répertoires donnée \$HOME/projets, écrivez une commande qui permet de trouver tous les fichiers .py et d'afficher la liste des fichiers les plus récents, en les triant par date de modification (du plus récent au plus ancien). Limitez l'affichage aux 5 fichiers les plus récents. (Utilisez les commandes `read`, `stat`, et `sort`).

Exercice 2

Écrivez un script shell `[votre-nom].sh` qui prend en arguments deux fichiers texte (`file1` et `file2`) ainsi qu'un fichier de sortie `result`. Le script doit réaliser plusieurs opérations avancées de traitement et d'analyse de texte. Deux fonctions prédéfinies dans le script ci-dessous (`tools.sh`) sont fournies pour vous aider dans ce traitement.

```
x () {
    sed -e 's/^[ \t]*//; s/[ \t]*$//' "$1" | sed '/^$/d'
}

y () {
    while read -r ligne; do
        echo "$(echo -n "$ligne" | wc -c): $ligne"
    done < "$1" | sort -n -t ':' -k1 | cut -d ':' -f2-
}
```

Listing 1: tools.sh

Le script suivra les usages standards d'Unix. Son comportement est le suivant:

1. **Vérification des arguments :**

Le script doit être exécuté avec trois arguments : `file1`, `file2`, et `result`.

Si le nombre d'arguments est incorrect, affichez un message d'erreur et terminez le script avec un statut d'échec.

2. **Vérification de l'accessibilité des fichiers :**

Assurez-vous que `file1` et `file2` sont accessibles en lecture, et que `result` est accessible en écriture. Si l'un de ces fichiers n'a pas les permissions adéquates, affichez un message d'erreur et terminez le script.

3. **Fusion, nettoyage, et tri des fichiers :**

Indiquez ce que font les fonctions fournies `x` et `y`. Lisez les fichiers `file1` et `file2`, appliquez ces deux fonctions pour nettoyer les lignes et trier les lignes nettoyées par longueur. Écrivez le résultat dans le fichier `result` dans une section intitulée **Tri des fichiers**.

4. **Extraction de lignes contenant un mot spécifique :**

Demandez un entier `N` à l'utilisateur sur le flux standard d'entrée (`stdin`), et vérifiez que $0 < N < 5$. Demandez un mot-clé à l'utilisateur en utilisant l'entrée standard (`stdin`). Recherchez toutes les lignes de `file1` et de `file2` contenant ce mot-clé, sans tenir compte de la casse, et ajoutez-les répétées `N` fois sous une section **Lignes contenant le mot-clé** : dans le fichier `result`.

5. **Calcul de statistiques sur les longueurs de lignes :**

Calculez les statistiques suivantes sur les longueurs des lignes de `result`:

- La **longueur minimale** des lignes dans les fichiers `file1` et de `file2`.
- La **longueur maximale** des lignes dans les fichiers `file1` et de `file2`.
- La **longueur moyenne** des lignes dans les fichiers `file1` et de `file2`.

Ajoutez ces statistiques au fichier `result` dans une section intitulée **Statistiques des longueurs de lignes**.

6. **Extraction de lignes :**

Lisez un entier `A` et un entier `B` sur le flux standard d'entrée (`stdin`), et vérifiez que $0 < A < B$, sinon le script affiche un message d'erreur et échoue. Écrivez dans le fichier `result` dans une section intitulée **Extraction de lignes** les informations suivantes:

- le contenu des lignes de `file2` en remplaçant (sans tenir compte de la casse) toutes les occurrences de l'URL `https://www.universite-paris-saclay.fr/` par `https://www.ensiie.fr/`, et `FISE` par `FISA`, et ce en utilisant une seule commande shell.
- le contenu des lignes commençant par le mot `POSIX` (sans tenir compte de la casse), allant de la ligne numéro `A` à la ligne numéro `B` de `file2`.

- les lignes non vides de `file2`, excluant les lignes allant de la ligne numéro `A` à la dernière ligne de `file2`.

7. Affichage du fichier de résultats :

Affichez le contenu de `result` sur la sortie standard *stdout*.

8. Archivage du fichier de résultats :

Créez une archive compressée `[votre-nom].tar` contenant le fichier `result`.

Complétez le script `[votre-nom].sh` pour qu'il réalise toutes les tâches décrites, **en utilisant** les deux fonctions du script `utils.sh`, et en créant d'autres au besoin.